Directus AI + MCP — Complete Client Documentation

Project: Functional — Directus CMS (headless CMS for the Functional Next.js website) **Date:** June 2026
Version: 1.0

> ## ⚠ Current Status — Read First
>
> The built-in Directus MCP server requires Directus **v11.12.0 or higher**.
>
> - **Staging — ✅ ready.** We have **already upgraded the Staging environment to v11.17.4** (the latest stable version), so MCP can be enabled there now.
> - **Production — ❌ not yet.** Production still runs **v11.11.0**, which is **below** the minimum and must be upgraded to v11.12.0 or newer before MCP can be used there.
>
> ** Action for your DevOps team:** Please **update the Directus version in the environment configuration** to match the upgrade we applied on Staging (v11.17.4), and apply the same upgrade to Production when ready.
>
> Everything described in this document regarding MCP capabilities is available on an environment **once it is on ≥ v11.12.0**. We can perform the Production upgrade and the MCP configuration for you — see Sections 1 and 3.

Table of Contents

1. Environment Overview — Current Status

1.1 Functional Environments

Environment	URL	Directus Version	MCP Supported
Staging	https://directus.staging.functional.team	**11.17.4**	✅ Yes
Production	https://directus.functional.team	**11.11.0**	❌ Not yet

Functional has **two** environments — **Staging** and **Production**.

- **Staging** has been **upgraded to v11.17.4** (the latest stable version) and now meets the MCP requirement — MCP can be enabled there immediately.
- **Production** still runs **v11.11.0**, which is **below** the minimum required for the built-in MCP server.

> **To use MCP on Production, it must first be upgraded to v11.12.0 or newer** (we recommend matching Staging at v11.17.4). See Section 3.
>
> ** DevOps note:** Please update the Directus version in the environment configuration to reflect the Staging upgrade (v11.17.4).

1.2 MCP Minimum Version Requirement

The built-in MCP server was introduced in Directus **v11.12.0**. Any instance below this version cannot use the native MCP functionality. **Staging** (v11.17.4) now meets this requirement; **Production** (v11.11.0) still falls short and requires an upgrade.

2. What is MCP and How it Connects to Directus

> The capabilities in this section become available on Functional **only after** the upgrade in Section 3. They are described here so you understand what MCP delivers and why the upgrade is worthwhile.

2.1 What is MCP?

MCP (Model Context Protocol) is an open standard developed by Anthropic that allows AI assistants like Claude to connect directly to external tools and systems — not just read about them, but actually interact with them in real time.

Think of it as a **structured API bridge** between the AI and your Directus instance:

```
Claude (AI) ←——— MCP Protocol ———→ Directus MCP Server ←——→ Directus Database
```

Without MCP (e.g. Production today on v11.11.0, or any environment where MCP is not enabled), AI assistance still works but is manual:

- You copy-paste data from Directus into the chat
- You describe your schema manually to the AI
- The AI generates code or step-by-step instructions you then execute yourself

With MCP (after upgrading to ≥11.12.0), Claude can:

- Query your live database directly
- Read schema structures automatically without any manual description
- Create, update, and manage content
- Build and modify automation flows
- Trigger existing workflows on your behalf

2.2 How MCP Connects to Directus

The Directus MCP server exposes a set of **tools** (functions) that Claude can call. The connection flow is:

```
1. Claude Code / Claude Desktop starts
2. Reads MCP configuration (settings.json or inline config)
3. Connects to the Directus MCP server endpoint
4. MCP server authenticates to Directus using your Static Token or OAuth
5. Claude can now call MCP tools as part of any conversation
```

The MCP server is the **official Directus MCP package** (`@directus/mcp`) — either the built-in server (Directus 11.12+) or the standalone npm package.

2.3 Tool Naming Convention

Once connected, Claude gains access to a set of named tools. The prefix is the name you give the connection in your configuration. For example, if you name the connection `directus-staging`, all tools

are prefixed:

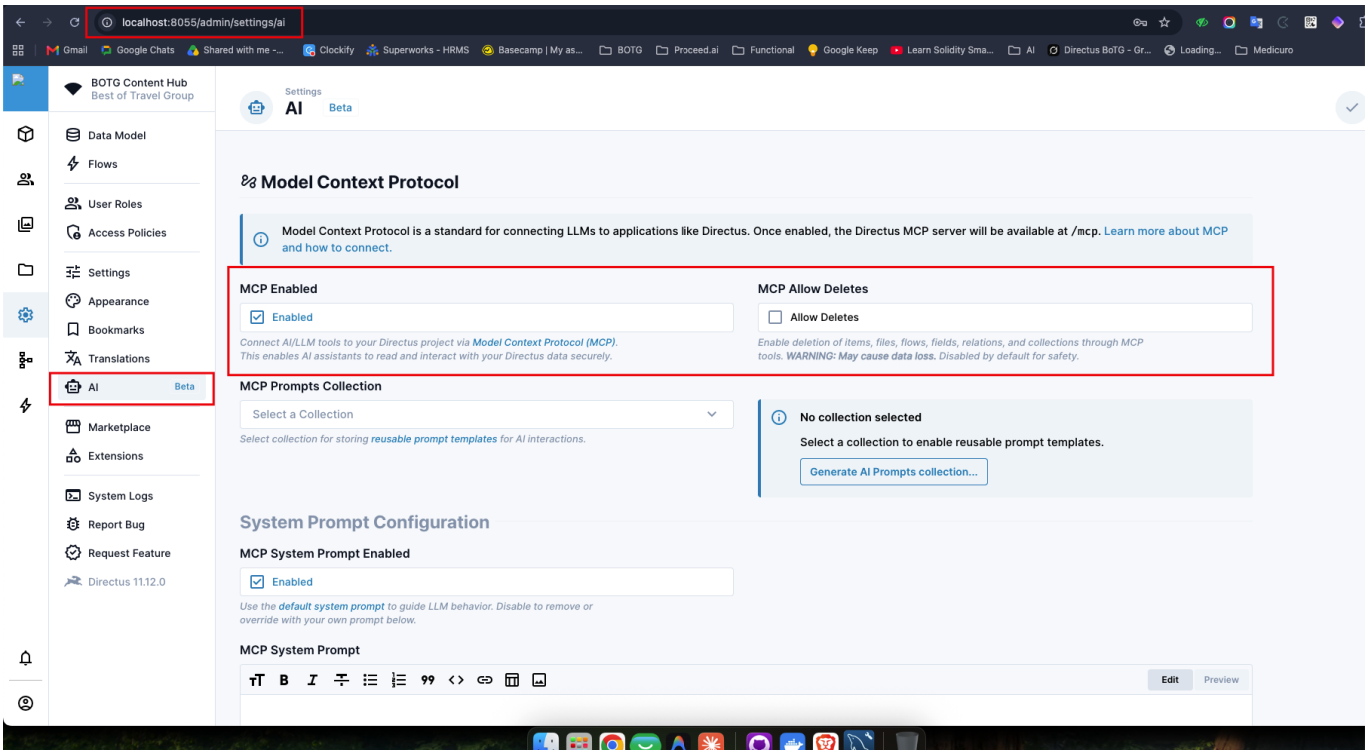
- `mcp__directus-staging__schema`
- `mcp__directus-staging__items`
- `mcp__directus-staging__collections`
- `mcp__directus-staging__fields`
- `mcp__directus-staging__flows`
- `mcp__directus-staging__trigger-flow`
- (and more — see Section 6)

2.4 Quick Reference — Enabling MCP and Connecting a Client

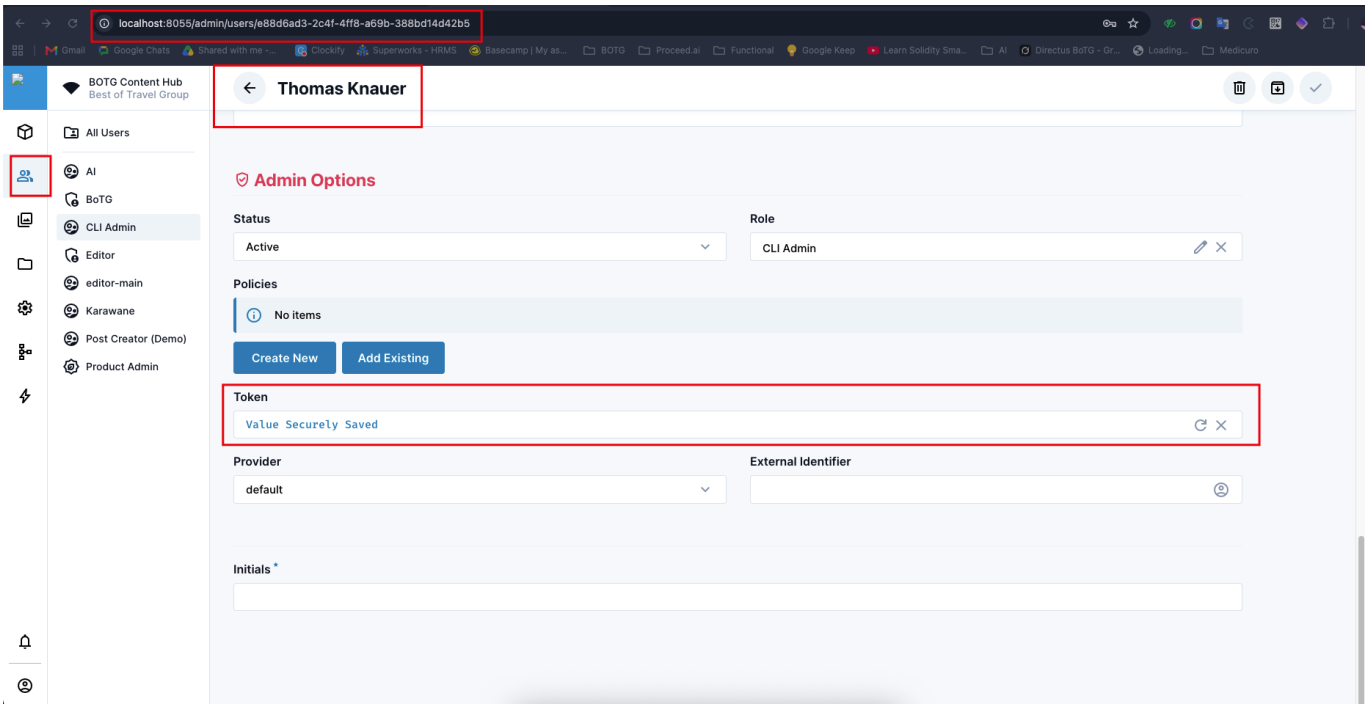
This is the short end-to-end path **(valid once Functional is on ≥11.12.0)**. Detailed instructions follow in Sections 4 and 5.

Enabling MCP — for each environment where MCP should be available:

1. Navigate to `/admin/settings/ai`
2. Enable the MCP server (toggle **MCP Enabled**)
3. Open the desired user account (e.g. *Thomas Knauer*)
4. Generate an MCP access token for that user
5. Copy and securely store the generated token — it will be required by the AI client



Directus → Settings → AI → Model Context Protocol. The "MCP Allow Deletes" option is disabled by default — it must be explicitly enabled to permit deletion of items, files, flows, fields, relations, and collections through MCP tools.



*Open the user account (Settings → Users), generate a token in the **Token** field, then save. The value is shown only once at generation time and afterward displays as "Value Securely Saved".*

Connecting an AI Client

Directus provides setup instructions for various AI clients:

- https://directus.com/docs/guides/ai/mcp#supported-clients
- https://directus.com/docs/guides/ai/mcp/installation#connect-your-ai-client

For example, with **Claude Code**:

OAuth (Recommended)

```
claude mcp add --transport http directus https://your-directus-url.com/mcp
```

Then start Claude Code and complete the browser authorization flow.

Static Access Token (if OAuth is not available)

```
claude mcp add --transport http directus https://your-directus-url.com/mcp
\
  --header "Authorization: Bearer your-generated-token"
```

Once configured, the AI client will be able to interact with the Directus instance through MCP using the **permissions of the user associated with the generated token**.

> If you would like us to perform the Functional upgrade or assist with the MCP configuration, please let us know.

3. Prerequisite — Upgrading Functional to Support MCP

This is the **gating step** for everything MCP-related on Functional. **Staging is already done; Production is pending.**

3.1 The Requirement & Current State

Item	Value
Minimum for built-in MCP	**11.12.0**
Staging	**Upgraded → v11.17.4** ✅ — meets requirement
Production	**v11.11.0** — below minimum, upgrade required
Action required	Upgrade **Production** to **v11.12.0 or newer** (recommend matching Staging at **v11.17.4**)

> **DevOps action:** The repo `directus/package.json` is still pinned to `11.11.0`. Please **update the Directus version in the environment configuration** to match the Staging upgrade (v11.17.4).

3.2 Recommended Upgrade Path

> Steps 1–3 are **already complete on Staging** (now on v11.17.4). They are documented here for the Production upgrade.

1. **Bump the version** in `directus/package.json` (to the latest stable 11.x release, ≥ 11.12.0 — Staging uses **v11.17.4**) and update the lockfile.
2. **Apply on Staging first.** Redeploy with the new version and run `npm run migrate` (`directus database migrate:latest`). *(Done — Staging is on v11.17.4.)*
3. **Verify Staging** — confirm the admin panel loads, existing collections/fields/flows are intact, the Next.js frontend still renders, and `/admin/settings/ai` shows the **Model Context Protocol** section. *(Done.)*
4. **Promote to Production** — repeat the same version-bump deploy on Production after Staging is validated. **← next step.**
5. **Enable MCP** per Section 4, then connect Claude per Section 5. (Can be done on Staging now; on Production after step 4.)

> **Safety note:** Take a database backup of each environment before upgrading. Review the Directus release notes between 11.11.0 and the target version for any breaking changes, especially anything affecting the headless API the Next.js frontend depends on.

We can perform the Production upgrade on request.

---

4. Enabling MCP on Your Directus Instance

> Available on **Staging now** (v11.17.4). On **Production** after its upgrade (Section 3).

4.1 Step-by-Step: Enable MCP in the Directus Admin Panel

1. Log into your Directus Admin panel
2. Navigate to **Settings** (gear icon in the sidebar)
3. Select **AI** from the left menu
4. Scroll down to the **Model Context Protocol** section
5. Toggle **Enable MCP** to ON
6. Click **Save**

> **Safety setting — MCP Allow Deletes:** On the same screen there is a separate **MCP Allow Deletes** toggle, which is **disabled by default**. While disabled, MCP tools cannot delete items, files, flows, fields, relations, or collections — they can only read, create, and update. We recommend leaving this OFF on Production and enabling it only in Staging when a deletion task is genuinely required, then disabling it again. This is the primary guardrail against accidental data loss via AI.
>
> There is also an **MCP System Prompt** section where a default (or custom) system prompt can be configured to guide the AI's behavior, and an optional **MCP Prompts Collection** for storing reusable prompt templates.

4.2 Generate a Directus MCP Access Token

The MCP server authenticates using a **Static Token** — a permanent, non-expiring credential tied to a specific Directus user account.

Step-by-Step:

1. Log into your Directus Admin panel
2. Navigate to **Settings → Users**
3. Open the user account you want the AI to act as
   - **Best practice:** Create a dedicated "AI Agent" user (e.g. `ai-agent@functional.team`) rather than using a personal account — this creates a clear audit trail
4. Scroll to the **Token** section
5. Click **Generate Token** (or type a custom value)
6. **Copy the token** — it is only shown once
7. Click **Save**

> **Important:** Store the token securely. Never commit it to Git. Use environment variables or a secrets manager.

4.3 Token Per Environment

Use a **different token for each environment** (Staging, Production). This ensures that a Staging breach does not affect Production, and that you can revoke access per environment independently.

---

5. Connecting an AI Client (Claude)

> Available on **Staging now** (v11.17.4). On **Production** after its upgrade (Section 3).

5.1 Method 1 — OAuth (Recommended)

For Claude Code, run this command and complete the browser authorization flow:

```
claude mcp add --transport http directus
https://directus.staging.functional.team/mcp
```

Then start Claude Code and you will be redirected to authorize via the Directus admin login.

5.2 Method 2 — Static Access Token

If OAuth is not available or you prefer a static configuration:

```
claude mcp add --transport http directus
https://directus.staging.functional.team/mcp \
  --header "Authorization: Bearer your-generated-token"
```

5.3 Method 3 — Configuration File (Claude Code mcp_settings)

In your project's `.claude/settings.json` (or `~/.claude/settings.json` for global configuration), add an `mcpServers` block:

```
{
  "mcpServers": {
    "directus-staging": {
      "command": "npx",
      "args": ["-y", "@directus/mcp"],
      "env": {
        "DIRECTUS_URL": "https://directus.staging.functional.team",
        "DIRECTUS_TOKEN": "your-static-token-here"
      }
    }
  }
}
```

The key (`"directus-staging"`) becomes the prefix for all tool names.

5.4 Connecting Both Environments Simultaneously

You can connect both Functional instances at the same time — Staging and Production. Claude will have tools for both connections:

```
{
  "mcpServers": {
    "directus-staging": {
      "command": "npx",
      "args": ["-y", "@directus/mcp"],
      "env": {
        "DIRECTUS_URL": "https://directus.staging.functional.team",
        "DIRECTUS_TOKEN": "staging-token-here"
      }
    },
    "directus-production": {
      "command": "npx",
      "args": ["-y", "@directus/mcp"],
      "env": {
        "DIRECTUS_URL": "https://directus.functional.team",
        "DIRECTUS_TOKEN": "production-token-here"
      }
    }
  }
}
```

5.5 Supported AI Clients

Directus MCP works with any MCP-compatible AI client. Confirmed supported clients include:

- **Claude Code** (CLI — primary tool used in this project)
- **Claude Desktop** (Mac/Windows app)
- Any MCP-compatible client that supports HTTP transport

Reference documentation:

- https://directus.com/docs/guides/ai/mcp#supported-clients
- https://directus.com/docs/guides/ai/mcp/installation#connect-your-ai-client

---

6. What IS Possible via MCP — Capabilities

> These are the tools the Directus MCP server exposes on an MCP-enabled environment (**Staging now**; Production after its upgrade). Examples below use generic website-CMS collections (pages, posts, categories, navigation).

6.1 Schema Discovery

The AI can fully explore and understand the database structure **without any manual explanation**.

What it does:

- Lists all collections and collection folders
- Returns field types, validation rules, notes, and display interfaces
- Reads all relationship maps: M2O, O2M, M2M, M2A, and Translation junctions
- Understands field groups, accordions, and alias field configuration

Example prompt:

> "What fields does the `pages` collection have?" → AI calls the schema tool, reads all field definitions, and answers accurately without guessing.

6.2 Data Operations — Items (Full CRUD)

Full Create, Read, Update, Delete on any collection the token has access to.

Operation	Capability
Read	Filter, sort, paginate, search, aggregate, deep-query nested relations
Create	Single or batch, with nested relation creation
Update	Single or batch, partial updates
Delete	By primary key(s)

Supported filter operators: `_eq`, `_neq`, `_in`, `_nin`, `_icontains`, `_starts_with`, `_between`, `_lt`, `_gt`, `_null`, `_nnull`, `_some` (O2M), `_none` (O2M), `_and`, `_or`

Deep queries: Can filter and sort nested relationships in a single call.

Aggregation: `count`, `sum`, `avg`, `min`, `max` — with `groupBy` support.

Example prompts:

- "Show me all `pages` with status = published"
- "Create a new blog `post` with translations for the configured languages"
- "Update the `meta_description` on all `pages` in the 'Services' category"
- "Count how many `posts` exist per category"

6.3 Collections Management

The AI can design and create the entire database structure.

- Create new collections with full metadata (icon, color, note, sort field, archive config, display templates)
- Configure singleton collections (for site settings / global records)
- Enable content versioning
- Create collection folders for UI grouping
- Update or delete collections

Example: "Create a `blog_posts` collection with UUID primary key, title, slug, content, published_at, and a status field with archive support."

6.4 Fields Management

Full field CRUD with complete interface and display configuration.

All field types supported:

- Text: `string`, `text`, `uuid`, `hash`
- Numeric: `integer`, `bigInteger`, `float`, `decimal`
- Date/Time: `timestamp`, `datetime`, `date`, `time`
- Boolean, JSON, CSV
- Geospatial: `point`, `lineString`, `polygon`
- Alias (virtual): for relationship display, groups, accordions, flow triggers

Full interface configuration — the AI sets the correct Directus interface for each field (e.g. `input-rich-text-md`, `select-dropdown-m2o`, `list-o2m`, `map`, `input-multiline`, `collection-item-dropdown`, the WYSIWYG editor) and all display options including **notes**, **labels/translations on the field**, conditions, validation, and readonly flags.

6.5 Relations Management

The AI can design and create all Directus relationship types:

Type	Description	Example for a website CMS
M2O	Many-to-One	`post → author`, `page → category`
O2M	One-to-Many	`page → sections`, `category → posts`
M2M	Many-to-Many (junction)	`posts ↔ tags`, `pages ↔ related_pages`
M2A	Many-to-Any (polymorphic)	Page builder blocks pointing to multiple block types
Translations	Built-in i18n junction	`pages_translations`, `posts_translations`

6.6 Files & Media

Files tool:

- Read file metadata (title, type, filesize, dimensions, folder, upload date, tags)
- Filter files by type, folder, size, date
- Update metadata in bulk (titles, tags, descriptions, focal points, alt text)
- Import files from external URLs directly into Directus
- Delete files by ID

Assets tool:

- Retrieve actual file content as base64 (images and audio)
- Enables the AI to **visually analyze images** stored in Directus
- Useful for: checking image quality, reading text in images, generating alt text, validating content

Folders tool:

- Create, read, update, delete virtual folders
- Build folder hierarchy for media organization

6.7 Automation Flows

The AI can fully read, create, and modify the automation system.

Flow trigger types the AI can work with:

Trigger	Description
`event`	Fires on database events (create/update/delete on specific collections)
`schedule`	Cron-based scheduling (daily, weekly, etc.)
`webhook`	External HTTP trigger
`manual`	Button in the Directus UI or triggered via API
`operation`	Sub-flow called by another flow

Operation types available:

- `condition` — branching logic
- `exec` — run arbitrary JavaScript (Node.js)
- `request` — HTTP requests to external APIs
- `log` — debug logging
- `send-email` — email notifications (Functional has SMTP configured)
- `send-notification` — in-app Directus notifications
- Read/write/update/delete items within a flow

Useful flow ideas for a headless website: revalidate/rebuild the Next.js frontend on publish (webhook to the deployment), auto-generate slugs, auto-fill SEO metadata, image alt-text generation on upload, scheduled publishing/unpublishing, editor notifications on publish-date.

6.8 Triggering Flows Programmatically

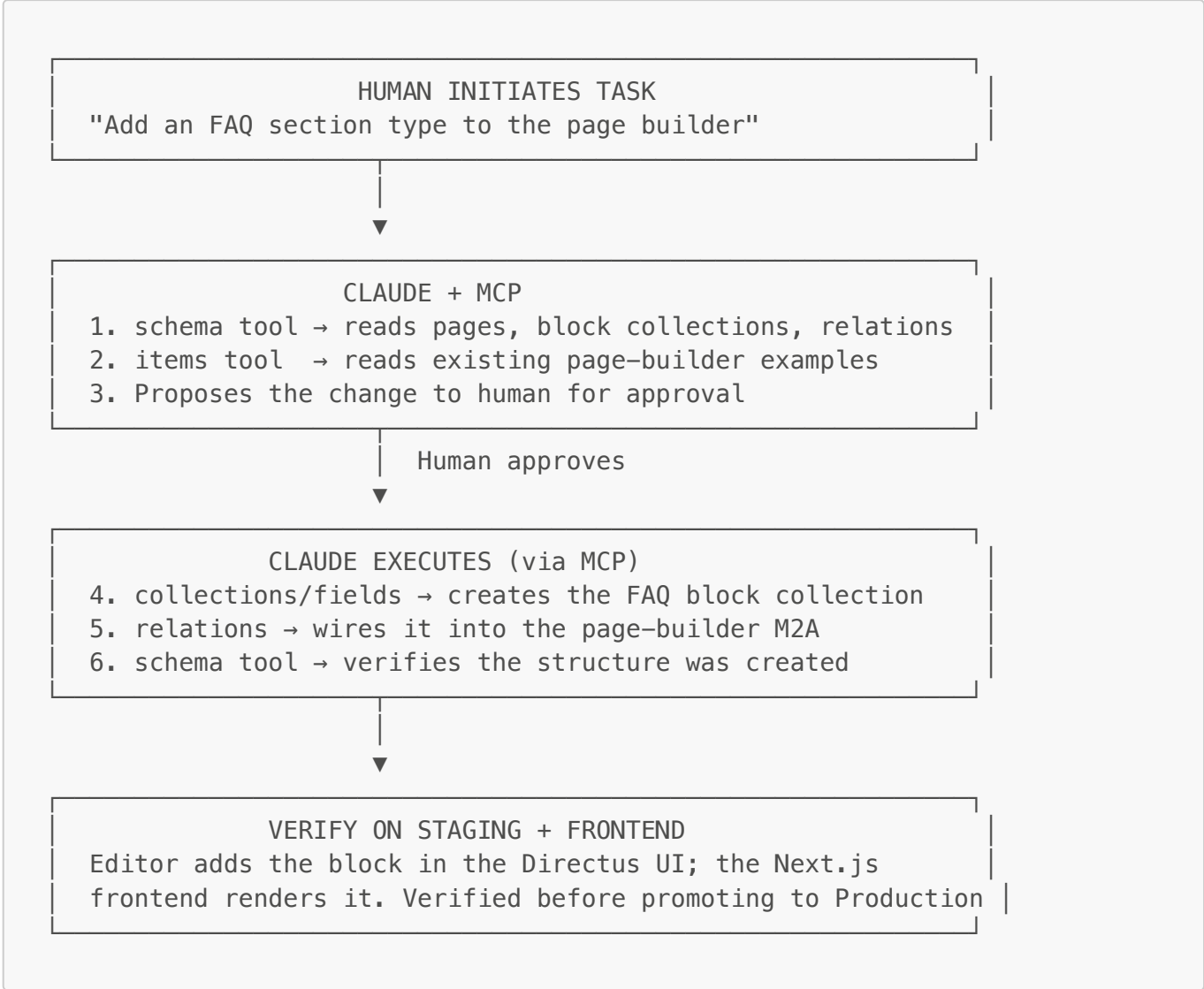
The AI can execute **manual flows** directly, passing data payloads and collection context.

Example: Trigger a "Translate page content" flow on a selection of page IDs, or trigger a "Revalidate frontend" flow after a batch content update.

---

7. Our AI Workflow in Practice

7.1 Standard Workflow — Human Stays in Control



Core principle: AI always reads the schema before writing anything. Every significant change is proposed to the human before execution. Destructive operations (deletes, bulk updates) are always confirmed first. Because the site is headless, changes are verified on Staging and against the Next.js frontend before reaching Production.

7.2 Example AI-Assisted Tasks for Functional

These are representative of how AI + MCP would be used on a headless website CMS like Functional (available on Staging now; on Production after its upgrade):

- **SEO metadata fill** — AI reads page content and drafts `meta_title` / `meta_description` / Open Graph fields across pages for editor review.
- **Image alt-text generation** — using the assets tool, AI visually analyzes uploaded images and writes accessible `alt` text into the file metadata.
- **Content translation drafts** — AI batch-translates translatable fields into the configured target languages; editors review and approve.

- **Page-builder scaffolding** — AI creates new block collections and wires them into the page builder (M2A), following existing block patterns.
- **Frontend revalidation flow** — AI builds a flow that pings the Next.js deployment to rebuild/revalidate when content is published.
- **Bulk content cleanup** — AI finds and proposes fixes for broken links, missing slugs, or empty required SEO fields (human confirms before write).

---

8. Creating Collections, Fields, Translations & Flows via AI

This section describes how we use Claude + MCP to build Directus schema from scratch (on any environment on ≥11.12.0 — Staging is ready now). These are the practices we follow.

8.1 Naming Conventions

All schema work follows strict naming conventions:

Element	Convention	Example
Collections	Plural, snake_case	`pages`, `posts`, `posts_translations`
Fields	Singular if one, plural if many; snake_case	`author`, `categories`, `published_at`
Junction tables	`{collection_a}_{collection_b}`	`posts_tags`, `pages_directus_files`
Translation tables	`{collection}_translations`	`pages_translations`
UI Labels	Title Case for nouns, lowercase for articles	"Meta Description", "of", "and"
Content languages	Use the project's configured locales	(defined in the `languages`/`translations` collection)

8.2 Creating a Collection

Prompt pattern:

> "Create a `posts` collection with UUID primary key, status field with archive support, sort field, user_created/date_created/user_updated/date_updated audit fields."

Claude will:

1. Read existing similar collections for reference (e.g. `pages`)
2. Create the collection with the correct meta configuration
3. Add all specified fields with correct types and interfaces
4. Confirm the creation and show a summary

What gets configured per collection:

- Primary key type (UUID or integer)
- Sort field for manual reordering
- Status field with `archive_field`, `archive_value`, `unarchive_value`
- Display template (what appears in relation dropdowns)
- Icon and color for the UI sidebar
- Collection folder for grouping

8.3 Adding Fields (with Notes & Labels)

Prompt pattern:

> "Add a `category` field to `posts` — M2O to the `categories` collection, required, with the note 'Primary category for this post' and the label 'Category'."

Claude will:

1. Read the schema of the target collection and the related collection
2. Create the field with correct type, interface (`select-dropdown-m2o`), and display
3. Create the relation automatically
4. Apply the field **note** and **label** (and translated field labels where configured)

Notes, labels, and translations on fields are part of the field's `meta` configuration, all of which the AI sets directly:

- **Note** — the helper text shown beneath a field in the editor
- **Label / field name display** — the human-readable name
- **Translations** — Directus supports translating field labels and notes per admin UI language; the AI can populate these via the field's translation meta

Common field creation examples for a website CMS:

Field Type	Example
String input	`title`, `slug`, `meta_title`
Text / WYSIWYG	`body`, `excerpt`, `meta_description`
Integer	`reading_time`, `sort`
Boolean	`featured`, `show_in_nav`
Date	`published_at`, `event_date`
JSON	`page_builder` (repeater/blocks), `seo`
M2O relation	`category → categories`, `author → authors`
File / Image	`hero_image → directus_files`
Readonly	Any field populated by a flow (with the correct bypass)

8.4 Setting Up Translations (i18n)

Directus translations follow a junction-table pattern. For a `pages` collection with translatable content:

1. **Create the junction collection** `pages_translations`:
   - `id` (integer, auto-increment PK)
   - `pages_id` (FK → pages, ON DELETE SET NULL)
   - `languages_code` (FK → languages/translations)
   - All translatable fields (e.g. `title`, `body`, `meta_description`)
2. **Create the M2M relation** linking `pages` to the languages collection via the junction
3. **Add an alias field** on `pages` (type: alias, interface: `translations`)

4. **Content languages used:** whatever locales Functional has configured in its languages collection

Example alias field config:

```
Field: translations
Interface: translations
Related collection: pages_translations
Languages collection: languages
Language indicator field: code
```

The standard `translations` interface gives editors a per-language tab set for entering each locale's content. (If an AI-translation extension is added later — see Section 9 — a one-click "AI Translate" button can be layered on top.)

8.5 Creating Relations

M2O (Many-to-One):

> "Add `category` to `posts` as M2O to `categories`, ON DELETE SET NULL."

Claude creates the field + the relation in one step.

O2M (One-to-Many):

> "Add an alias field `sections` to `pages` pointing to the `page_sections` collection via `page_id`."

M2M (Many-to-Many with junction):

> "Create a M2M between `posts` and `tags`, junction table `posts_tags`, both FKs with ON DELETE CASCADE."

Claude creates the junction table, both FK fields, both relations, and the alias fields on each side.

8.6 Creating Automation Flows

Prompt pattern:

> "Create a flow that triggers on `pages` create and update events and sends a webhook to the Next.js deployment to revalidate the affected page."

Claude will:

1. Read existing similar flows for reference
2. Design the operation chain
3. Create the flow with the correct trigger
4. Create each operation in order, wiring them together
5. Verify the chain by reading the created flow back

Flow architecture patterns we use:

- **Event + sub-flow pattern:** CREATE and UPDATE event flows fan out individual IDs to a TRIGGER sub-flow that does the heavy work. Prevents timeouts and allows parallel processing.
- **Parallel fan-out:** `iterationMode: "parallel"` on the trigger operation runs the sub-flow once per item ID simultaneously.
- **Permissions bypass:** Operations that write to readonly fields use `permissions: "$trigger"` and `emitEvents: false` to bypass UI readonly flags and prevent event loops.
- **Schema-first:** AI always reads the schema of all involved collections before creating any flow.

---

9. Custom Extensions — How They Work with AI

> **Functional currently has no custom Directus extensions.** This section explains how extensions work and how AI assists with them **if and when** Functional adds custom UI or server-side behavior (e.g. a custom interface, an AI-translation button, a page-builder preview, or a custom endpoint).

9.1 What Extensions Are

Directus extensions are **compiled JavaScript bundles** that Directus loads at startup. Any custom admin-interface component, custom module, custom endpoint, or hook is a custom extension.

Extensions live in the Directus `extensions` directory and are built during deployment. Directus scans the extensions directory at startup and loads every extension it finds.

9.2 Extension Technology Stack

Layer	Technology
UI Components	Vue 3 Composition API (`<script setup lang="ts">`)
Language	TypeScript
Build Toolchain	Directus Extensions SDK (Vite-based)
Native UI	Directus components (`<v-button>`, `<v-icon>`, `<v-dialog>`, etc.)
Theming	Directus CSS variables (`var(--theme--primary)`, etc.)

Build output per extension:

- `dist/app.js` — the compiled Vue component tree, loaded by the browser
- `dist/api.js` — the compiled server-side code (endpoints and hooks)

9.3 The Build and Deploy Cycle

AI can write, refactor, and fix extension code, but it cannot build or deploy them through MCP. Every extension change requires:

```
1. Developer/AI edits source (.vue / .ts files)     ← AI can assist here
          │
          ▼
2. npm run build  (inside the extension directory)   ← runs locally or in
the build step
          │                                             MCP CANNOT DO THIS
          ▼
3. dist/app.js + dist/api.js produced                ← build output is
ready
          │
          ▼
4. Redeploy / restart Directus                       ← loads the new
bundle
          │                                             MCP CANNOT DO THIS
```

```
          ▼
5. Browser hard refresh                              ← client clears cache
to get new bundle
          │
          ▼
6. Extension change is live
```

> Directus loads extensions only at startup, so a redeploy or restart of the Directus instance is what activates a new or changed extension bundle.

9.4 Why MCP Cannot Update Extensions at Runtime

Reason 1 — MCP has no shell access. MCP tools operate entirely through the Directus REST API. They cannot execute shell commands (`npm run build`), write to the deployed filesystem, or restart processes.

Reason 2 — Source edits have zero effect without a build. Directus only reads the compiled `dist/app.js`, never the TypeScript or Vue source files.

Reason 3 — Extensions are loaded once at startup. Even after a new bundle is built, the running instance has already loaded the old version. A deploy/restart is required.

Reason 4 — Browser caching. The admin app downloads compiled extension bundles once. Even after a redeploy, the browser needs a hard refresh or cache bust to receive the updated bundle.

---

10. Critical Considerations Before Any Change — Directus Is a CMS, Not a Freeform App

This is the single most important section for anyone using AI (or working manually) on this project. **Directus is a Content Management System built on a relational database — it is not a custom application that can be freely rewritten.** Every collection, field, relation, and flow is bound by database rules and by dependencies on other objects. A change that looks isolated is rarely isolated.

For this reason, no change — whether made by AI via MCP, by a developer, or by the client in the admin UI — should ever be applied "blind." It must be preceded by analysis of the **current state**, the **intended new state**, and the **consequences and conflicts** between them.

10.1 Always Analyze Before You Change

Before creating, modifying, or deleting any collection, field, relation, or flow:

1. **Read the current state.** Use the schema tool to read the live structure of every collection, field, relation, and flow involved. Never assume a field name, type, or relationship — verify it against the live instance. (This is especially important on Functional because the client may have changed the schema in the UI since the last code sync.)
2. **Review what was changed previously.** Reconcile the new change with prior changes — including any made directly in the UI by the client — not an outdated mental model.
3. **Define the intended new state precisely.** Know exactly which objects will be added, modified, or removed, and how they should look afterward.
4. **Identify conflicts between current and new state.** For example: a field name that already exists, a relation that already binds the collection, a flow already listening on the same event, or a value that violates an existing constraint.
5. **Map the consequences.** Determine what else depends on the object — other collections, relations, flows, interfaces, **the Next.js frontend's queries**, and existing content rows.
6. **Propose, then confirm.** AI proposes the full plan with consequences spelled out; a human approves before anything is written. Destructive operations are always confirmed individually.

10.2 Why a Change Is Rarely Isolated — Database & Content Consequences

Because Directus sits directly on a relational database, schema changes have real, sometimes irreversible, effects on both structure and stored content:

Change	Possible consequence on DB / content
Deleting a field	Permanently drops that column and all data stored in it across every row. Cannot be undone without a backup.
Changing a field type	May fail, truncate, or corrupt existing values if the data is not compatible with the new type (e.g. text → integer).
Renaming a field	Breaks any flow, interface, display template and **frontend's API queries**, or other consumer that references the old name.
Deleting a collection	Drops the table and, depending on FK rules, cascades deletes or nullifies related rows in other collections.

Change	Possible consequence on DB / content
Changing a relation's ON DELETE rule	Alters what happens to child rows when a parent is deleted (CASCADE deletes them, SET NULL orphans them). Wrong choice causes silent data loss or orphaned records.
Adding a required field to a collection with existing rows	Existing rows may become invalid or block saves until backfilled.
Adding/editing a flow trigger	Can fire on existing automation events, create loops, or perform mass writes the moment it is saved.
Editing translation junctions	Can detach or duplicate per-language content rows if the junction or FK is misconfigured.

The key principle: **content and schema live in the same database.** You cannot change the schema without considering the content already stored against it, and on a headless site you cannot change either without considering the live frontend that reads it.

10.3 Ripple Effects — One Change Can Affect Many Others

- **Shared collections** (categories, authors, tags, languages, site settings) are referenced by many others. Editing one affects everything that depends on it.
- **Flow chains:** A change in one flow can break or skew downstream automation.
- **Junction and alias fields:** Adding or removing a relation often requires matching alias fields, junction tables, and `one_field` settings on the inverse side — miss one and the UI or API breaks.
- **The Next.js frontend:** Field renames, removed fields, or changed relation shapes can break frontend queries and break the live website.

Before any change, ask: what else reads from or writes to this object? AI must trace these dependencies (using the schema and flows tools) and surface them as part of the proposal, so the human understands the full blast radius before approving.

10.4 The Rules Directus Enforces

- **Snake_case, plural collections, singular/plural field naming** (Section 8.1) — deviating breaks consistency and tooling.
- **Translations require a junction table and an alias field** — you cannot just "add a translatable field."
- **Relations must define both sides** (the FK and, usually, the inverse alias / `one_field`).
- **Primary key types are fixed at creation** (UUID vs auto-increment integer).
- **System collections** (`directus_users`, `directus_roles`, `directus_permissions`, `directus_settings`) are managed by Directus and are intentionally not exposed to MCP.
- **Readonly fields** populated by flows must be written with the correct permission/event-bypass settings, or the write is rejected or triggers loops.

Working *with* these rules — not against them — is what keeps the instance stable. This is precisely why AI is instructed to read the schema first and follow existing patterns rather than invent new structures.

---

11. Migration Management — Environments & Deployments

11.1 Environment Strategy

Functional:

- **Staging → Production** (no separate Development environment)
- All schema and collection changes should start in **Staging**
- Never make direct schema changes in Production

Recommended workflow:

1. Make all schema, collection, field, and flow changes in Staging
2. Perform thorough testing and validation in Staging — including checking the Next.js frontend against Staging data
3. Deploy approved changes to Production only after verification

11.2 Why Directus Has No Native "Push to Production"

Directus does not provide a native "Push to Production" button or mechanism. This is a fundamental architectural limitation: each environment has its **own separate database**, and Directus has no built-in mechanism to selectively migrate individual changes between them.

The core challenges:

Challenge	Explanation
ID and UUID mismatch	The same record has different internal IDs in Staging vs Production
No change tracking	Directus does not track which changes were made since the last migration
Content conflicts	The same record may have been modified in both environments
Object dependency order	Collections, fields, relations, flows must be migrated in the correct dependency order
Selective migration	There is no way to choose "migrate only these 3 new fields" without a custom tool
Media assets	Files and folders have separate UUIDs and storage paths per environment
Roles and permissions	User roles and permission sets are not automatically synchronized

11.3 What a Custom Migration Tool Would Require

To build a proper "Push to Production" solution, the following would need to be designed and developed:

1. **Change detection** — compare schema, flows, content, and permissions between environments before deployment
2. **Conflict presentation** — show the user what is different and what would be overwritten
3. **Selective migration** — allow the user to choose what should and should not be migrated
4. **Conflict resolution** — handle cases where the same object was changed in both environments
5. **Rollback mechanism** — ability to undo a migration

6. **Object type coverage** — define which types of objects are included: collections, fields, flows, permissions, content, media, translations, settings
7. **Hosting and access** — decide where and how the tool runs (Directus extension, external application, CLI script)

This would be a **custom-built solution** with significant development effort. Scope and cost would need to be estimated based on the desired coverage.

11.4 Directus Native Schema Migration (Code-Based)

Independent of MCP, Directus provides a **schema snapshot/apply** mechanism and database migrations, which Functional already uses:

- `npm run migrate` runs `directus database migrate:latest` as part of deployment

This handles **core Directus version migrations** when the instance is deployed. It does **not**, on its own, move your custom collections/content selectively between environments — that is the gap the custom tool in 11.3 would fill. Directus's `schema snapshot` / `schema apply` CLI can export and import the **data model** (collections/fields/relations, not content) as a versioned file, which is a lighter-weight option worth considering for keeping Staging and Production schemas aligned.

11.5 Our Current Migration Approach (Manual, Documented)

In the absence of a native selective-migration tool:

For schema changes (collections, fields, relations):

1. All changes are made in Staging (via AI + MCP — Staging is on v11.17.4 — or via the UI)
2. Every change is documented (what was created, field-by-field details, relation definitions, flow UUIDs) with a **Revert Procedure**
3. Once verified in Staging, the same operations are repeated in Production
4. The AI reads the change documentation and replicates the exact same schema in Production

For flow changes: documented (operations chain, trigger config, all operation UUIDs) and replicated to Production.

For content/data changes: content is generally not migrated between environments; Production content is entered/edited directly in Production.

11.6 Snapshots and Backups

Directus does not provide a built-in full-environment snapshot and restore system. A complete backup/restore solution would require a custom implementation defining:

- What to include: content, schema, flows, permissions, media
- Backup frequency and retention period
- Full vs selective restore
- Conflict handling for changes made after the backup

Current safety nets on Functional:

- **Database-level backups** are the primary restore point. Take a database backup before any upgrade or risky change.
- For schema, the `directus schema snapshot` file plus the documented revert procedures provide a manual revert path.

---

12. Collaboration, Content Safety & Deployment — Policy and Assessment

This section sets out our assessment and recommended practices for collaboration, content safety, environment management, and deployment for the **Functional environments**. It directly addresses the client's ClickUp questions.

> These considerations apply to the Functional environments (Staging and Production). MCP is available on **Staging now** (v11.17.4); on **Production** it is gated on the upgrade in Section 3.

12.1 Where Content Can Be Edited So That It Is Never Lost

The available Functional environments are:

- **Functional:** Staging, Production

It is important to note that Directus does **not** provide a native mechanism to isolate individual content, schema, flow, or configuration changes during environment migrations.

Since each environment has its own database, migrations operate at the database level. During deployment, it is not possible to reliably identify and selectively preserve only certain content, collections, fields, flows, permissions, or other changes that were manually created in another environment.

To achieve selective, conflict-aware migrations, we would need to develop a custom comparison and migration solution that:

- Detects differences between environments before deployment
- Presents those differences to the user
- Allows selection of what should and should not be migrated
- Handles conflicts where the same records or structures have been modified in multiple environments

Additional considerations include:

- Where and how such a tool would be hosted and maintained
- When the comparison should be executed
- How to handle manual changes made directly in Production
- How to resolve conflicting changes between environments
- Which types of objects should be compared (content, collections, fields, flows, permissions, translations, etc.)

Depending on the desired scope and level of automation, we can evaluate whether a custom application or migration utility would be the most appropriate approach and estimate the required effort accordingly.

12.2 Where Collections Should Be Edited

Collections, fields, relations, and other schema changes can be managed through the **Data Model** section within Directus Settings.

To maintain a controlled workflow, we recommend:

- Making all schema and collection changes in **Staging**

- Performing thorough testing and validation (including against the Next.js frontend)
- Deploying approved changes to **Production** only after verification

This helps reduce the risk of unintended production changes. With Staging already on v11.17.4, AI + MCP can be our primary tool for applying schema changes — the AI reads the Staging environment, applies the changes, and documents everything with revert instructions before Production is touched.

12.3 Pushing Changes from Staging to Production

Directus does **not** provide a native "Push to Production" functionality.

While it is technically possible to build a custom deployment tool, there are numerous challenges and considerations:

- Determining which objects should be migrated (collections, fields, flows, permissions, content, media, etc.)
- Ensuring Staging and Production are sufficiently aligned before deployment
- Managing differences in IDs, UUIDs, and references between environments
- Handling content conflicts and overwritten changes
- Managing user roles, permissions, and policies
- Establishing secure communication between environments (SSH, APIs, or other methods)
- Defining rollback procedures in case of deployment issues

Because of these complexities, such functionality would require a custom solution, either as:

- A dedicated Directus extension
- An independent deployment application
- A CLI-based schema sync built on `directus schema snapshot` / `schema apply`

The exact effort would depend on the scope of deployment functionality required.

Current workaround: We replicate schema changes from Staging to Production manually but accurately — using documented change logs and (on Production, once it is upgraded) AI + MCP to repeat the exact same operations against the Production connection. This is auditable and reversible.

12.4 Snapshots / Backups with Revert Functionality

Directus does **not** provide a complete environment snapshot and restore system out of the box.

A custom solution would need to define:

- What should be included in backups: content, collections, fields, flows, permissions, media assets
- How frequently backups should be created
- How long backups should be retained
- Whether restores should be full or selective
- How to handle changes made after a backup was taken

One of the major challenges is **conflict management**:

- Important changes may have been made after the backup was created

- A full restore could unintentionally overwrite newer content
- Selective restores require object-level comparison and reconciliation

Similar to environment migration and push-to-production (12.1 and 12.3), a robust snapshot and rollback mechanism would require a custom extension or application.

Current fallback: database-level backups provide a last-resort restore point. For schema, the `directus schema snapshot` file plus documented revert procedures provide a step-by-step revert path.

12.5 Enabling Directus MCP / Working with Claude

MCP can be enabled and used with Claude or other MCP-compatible AI tools, **provided the Directus version and environment meet the necessary requirements.**

Minimum required version: Directus **v11.12.0** or higher (built-in MCP).

Current status — Functional:

- **Staging** (https://directus.staging.functional.team) — **upgraded to v11.17.4** ✅ — meets the requirement, MCP can be enabled now.
- **Production** (https://directus.functional.team) — running **v11.11.0** — below the minimum; must be upgraded to v11.12.0 or newer first (Section 3).

> **DevOps action:** Please update the Directus version in the environment configuration to match the Staging upgrade (v11.17.4).

To enable MCP (per environment, once it is on ≥ v11.12.0 — Staging is ready now):

1. Navigate to `/admin/settings/ai`
2. Enable the MCP server
3. Open the desired user account (e.g. Thomas)
4. Generate an MCP access token for that user
5. Copy and securely store the generated token

To connect Claude: see Section 5 (OAuth and static token methods). With **Claude Code**:

```
# OAuth (recommended)
claude mcp add --transport http directus
https://directus.staging.functional.team/mcp

# Static token (if OAuth not available)
claude mcp add --transport http directus
https://directus.staging.functional.team/mcp \
  --header "Authorization: Bearer your-generated-token"
```

Once configured, the AI client interacts with Directus using the **permissions of the user associated with the generated token**.

Please let us know if you would like us to perform the Functional upgrade or assist with the MCP configuration.

---

13. What AI Cannot Do — Limitations

Understanding limitations is as important as understanding capabilities. This section covers both hard technical limitations and reliability boundaries. (In addition, recall that MCP is available on **Staging** now (v11.17.4) but **not on Production** until its version upgrade.)

13.1 What MCP Cannot Do

Limitation	Explanation
Run on Directus < 11.12.0	The built-in MCP server does not exist below v11.12.0. Staging is on v11.17.4 (OK); Production (v11.11.0) must be upgraded first.
Upload binary files	The `files` tool can only import from a URL or update metadata. It cannot accept a file upload directly from the AI session. Files must be uploaded through the Directus UI or REST API.
Access to Directus UI	MCP interacts with the API layer only. It cannot see, click, or screenshot the admin interface.
User management (create/delete users)	The MCP tools do not expose `directus_users` CRUD — intentionally kept out for security.
Role and permission editing	`directus_roles` and `directus_permissions` are system-level resources not exposed by MCP.
Extension code execution	AI can read and write extension source code files on the local filesystem, but cannot install, build, or hot-reload extensions through MCP — that requires shell access and a build/deploy.
Raw database migrations / SQL	Raw SQL is not accessible. Schema changes go through the collections/fields/relations tools, which use Directus's own migration system.
Webhooks configuration	`directus_webhooks` for direct management is not exposed.
Global settings	`directus_settings` (project name, logo, auth settings) is not exposed.
Dashboard/Panel creation	Insights dashboards and panels are not manageable via MCP.
Real-time subscriptions	MCP is request-response based; it cannot subscribe to live data changes.
Build or deploy Directus	MCP has no shell access and cannot trigger builds, deploys, or restarts.

13.2 What AI Cannot Do Reliably (Even With MCP)

Limitation	Explanation

Limitation	Explanation
Guarantee correctness without schema check	AI should always read the schema before writing. If it skips this step, it risks guessing field names incorrectly — especially likely on Functional, where the client edits the schema in the UI directly.
Handle high-volume bulk operations safely	Modifying thousands of records in a single instruction is risky. Large bulk operations should be reviewed, batched, and confirmed by a human before execution.
Make business logic decisions	AI can implement logic it is told, but cannot decide what the right content structure, taxonomy, or editorial rule should be. Those decisions stay with the business.
Recover from bad deletes	If a delete is confirmed and executed, MCP cannot undo it. Directus has revisions, but reconstruction requires manual steps. We always confirm before deleting.
Work offline or in air-gapped environments	Claude requires an internet connection to the Anthropic API.
Read encrypted/hashed field values	Fields with type `hash` (passwords) are one-way and can never be read back — not by AI, not by anyone.
Guarantee translation quality	AI translation produces drafts. Human review is required for market-facing copy, legal text, and brand-sensitive content.
Replace human judgement on content	AI can draft, suggest, fill, and organize content, but editorial decisions, brand voice validation, and legal accuracy must be verified by humans.
Guarantee no impact on the live website	Functional is headless — schema/content changes can affect the live Next.js site. Always verify on Staging and have a revert plan before applying changes to Production.

14. Summary Capability Table

> The "MCP (with Claude)" column is available on **Staging** now (v11.17.4); on **Production** it requires the upgrade to ≥11.12.0 first.

Capability	MCP (with Claude)	AI (without MCP)
Read live database content	✅	❌
Understand schema automatically	✅	❌ (must be described manually)
Create/update content items	✅	❌
Create collections and fields	✅	❌
Create and manage flows	✅	❌
Trigger existing flows	✅	❌
Manage file metadata	✅	❌
Analyze image content visually	✅ (via assets tool)	❌
Upload binary files	❌	❌
Manage users/roles/permissions	❌	❌
Read/write extension source code	❌	✅ (via Claude Code filesystem)
Run `npm run build` for extensions	❌	✅ (via Claude Code shell)
Make editorial/business decisions	❌	Advisory only
Guarantee translation quality	❌	Draft quality only
Access Directus UI directly	❌	❌
Native push staging → production	❌	❌ (requires custom tool)
Native backup/restore	❌	❌ (requires custom tool / DB backup)

15. Security Best Practices

15.1 Token and Access Security

Practice	Why
Create a **dedicated AI user** (e.g. `ai-agent@functional.team`)	Audit trail shows "ai-agent" did it, not a human user
Assign **minimum required permissions** via a Role	AI can't accidentally delete things it shouldn't touch
Never commit the token to Git	Use environment variables or a secrets manager
Use **different tokens** per environment	A Staging breach doesn't affect Production
Store tokens in environment variables / a secrets vault	Not hardcoded in config files
Rotate tokens if an environment is compromised	Revoke immediately from the user record in Settings → Users

15.2 Example Role Permissions for a Read-Heavy AI Agent

```
Collections: Read (all)
Files:        Read, Update (metadata only)
Flows:        Read, Trigger (manual only)
Schema:       Read only
Users:        No access
Settings:     No access
```

For a write-enabled AI agent (used in Staging schema work):

```
Collections: Read, Create, Update, Delete
Fields:       Read, Create, Update, Delete
Relations:    Read, Create, Update, Delete
Items:        Read, Create, Update, Delete (per collection, restricted)
Flows:        Read, Create, Update, Delete, Trigger
Files:        Read, Create, Update
Schema:       Read
Users:        No access
Settings:     No access
```

15.3 What to Always Confirm Before Executing

These operations should always be reviewed and approved by a human before the AI executes them:

- **Delete any item, collection, field, relation, or flow**
- **Bulk update of more than 50 records**

- **Any operation on a Production environment**
- **Changes to fields the Next.js frontend depends on (slugs, statuses, relation shapes)**
- **Changes to flow trigger conditions**
- **Changes to permissions or role configurations**

15.4 Audit Trail

Because we use a dedicated AI user for MCP operations, every change made by the AI appears in Directus's activity log under that user account. This provides a clear audit trail distinguishing human edits from AI-assisted edits — useful on Functional where the client also edits directly in the UI.

Appendix A — Directus MCP Tool Reference

> Available on any environment running Directus ≥11.12.0 — **Staging (v11.17.4) now**; Production after its upgrade.

Tool	Purpose
`schema`	Read collection and field definitions
`items`	CRUD on any collection
`collections`	Create, read, update, delete collections
`fields`	Create, read, update, delete fields
`relations`	Create, read, update, delete relations
`files`	Read and manage file metadata
`assets`	Retrieve file content (images, audio) as base64
`folders`	Create and manage virtual folders
`flows`	Read, create, update, delete flows
`operations`	Read, create, update, delete flow operations
`trigger-flow`	Execute a manual flow with a payload
`system-prompt`	Returns project-specific AI instructions

Appendix B — Useful Directus AI Documentation Links

- MCP Overview: https://directus.com/docs/guides/ai/mcp
- Supported Clients: https://directus.com/docs/guides/ai/mcp#supported-clients
- Installation & Connection: https://directus.com/docs/guides/ai/mcp/installation#connect-your-ai-client
- Schema snapshot / apply (CLI): https://directus.com/docs/guides/cli

---

Appendix C — Environment Quick Facts

Item	Value
Staging (Directus)	https://directus.staging.functional.team — **v11.17.4** ✅ (MCP-ready)
Production (Directus)	https://directus.functional.team — **v11.11.0** (upgrade pending)
MCP minimum version	**v11.12.0** (upgrade required)

Item	Value
Directus	Self-hosted. Staging on **v11.17.4**, Production on **v11.11.0**. Repo `directus/package.json` still pinned `11.11.0` (update to match Staging). `npx directus start`
Frontend	Next.js, headless — reads Directus via API + tokens
Migrations	`npm run migrate` (`directus database migrate:latest`)
Environments	Staging, Production